Cargo Risk Pipeline — v2.0 Changelog

File: `cargo_risk_v2.py`
Previous version: `cargo_risk_pipeline.py` (v1.0)
Date: April 2026

Summary

Version 2.0 is a targeted simplification of the v1.0 pipeline. Three subsystems — the Google News API feed, the news verification layer, and the KNN geo-threat mapper — have been removed and replaced with a lightweight static threat table and a radial-basis scoring function. The core routing and ML components (Dijkstra, Yen's K-shortest, XGBoost) are unchanged. The result is a leaner, fully auditable pipeline with no external API dependencies and improved XGBoost R^2 (0.942 → 0.969).

What Was Removed

1. Google News API Feed

v1.0 components removed:

- `fetch_news_threats(query, max_articles)` function
- `GOOGLE_NEWS_API_KEY` and `GOOGLE_NEWS_ENDPOINT` constants
- `THREAT_KEYWORDS` dictionary (NLP keyword sets per threat category)
- `THREAT_SEVERITY` dictionary (per-category severity weights)
- `REGION_GEO_MAP` dictionary (region name → lat/lon lookup for news text)
- `classify_threat(text)` — keyword-matching NLP classifier
- `extract_geo(text)` — region name extractor from article text
- `_synthetic_threats()` — fallback synthetic news dataset
- `ThreatEvent` dataclass (title, description, lat, lon, threat_type, severity, published_at, source, verified, confidence)

Why removed: The news feed introduced an external API dependency, variable data quality, and NLP classification noise. It required an API key at runtime and fell back to hardcoded synthetic data anyway — making it redundant in the pipeline's current scope.

---

2. News Verification Layer

v1.0 components removed:

- `VerificationLayer` class (entire class)
  - `SOURCE_TRUST` dict — per-source credibility multipliers (Reuters=1.0, BBC=1.0, Al Jazeera=0.85, etc.)
  - `verify_threat_events(threats)` method — confidence scoring, recency bonus, severity plausibility check
  - `verify_route_risk(route, all_routes)` method — cross-route consistency flags
- `ThreatEvent.verified` and `ThreatEvent.confidence` fields (no longer needed)
- Verification step in `CargoRiskPipeline.initialise()`

Why removed: Verification logic was tightly coupled to the news feed's `ThreatEvent` objects. Without live news data, the layer had no input to verify. The warning generation logic (safer alternatives, chokepoint flags, war-zone adjacency) has been preserved and moved directly into `generate_warnings()` in v2.

---

3. KNN Geo-Threat Mapper

v1.0 components removed:

- `KNNGeoThreatMapper` class (entire class)
  - `fit(threats)` — fitted KNeighborsRegressor on threat event coordinates
  - `predict_threat_score(lat, lon)` — KNN inference at a point
  - `edge_threat_score(p1, p2, samples)` — max KNN score sampled along an edge
- `sklearn.neighbors.KNeighborsClassifier` and `KNeighborsRegressor` imports
- `sklearn.pipeline.Pipeline` import (was only used to wrap KNN + StandardScaler)
- `knn_geo_threat_score` field from `RoutePlan` dataclass
- KNN fitting step in `CargoRiskPipeline.initialise()`

Why removed: KNN was used to interpolate threat intensity from sparse news event coordinates. Removing the news feed eliminated the KNN training data entirely. The spatial threat estimation is now handled by the static threat table with a simpler and more transparent formula.

---

4. NetworkX Graph Builder

v1.0 components removed:

- `import networkx as nx`
- `build_graph()` function — constructed a `nx.DiGraph` with node attributes
- All `G.nodes[n]["geopoint"]` attribute lookups
- `G.successors(u)` traversal calls
- Node restoration bug workaround in Yen's algorithm (caused by NetworkX node attribute loss on re-add)

Why removed: NetworkX was only used as an adjacency container. A plain Python `dict` serves the same purpose with no dependency overhead and avoids the node-attribute-loss bug that required a patch in v1.0's Yen's implementation.

What Was Added

1. Static Threat Zone Table

New: `ThreatZone` dataclass and `THREAT_ZONES` list (12 pre-defined zones).

```
ThreatZone(name, lat, lon, threat_type, severity, radius_km)
```

Zone	Type	Severity	Radius
Red Sea — Houthi conflict	war	9.5	600 km
Strait of Hormuz — Iran risk	terrorism	8.8	350 km
Gulf of Aden — piracy	piracy	7.5	400 km
Gaza / E. Med conflict	war	7.8	300 km
Black Sea — Ukraine war	war	7.5	500 km
Persian Gulf — sanctions	sanctions	7.0	400 km
Gulf of Guinea — piracy	piracy	6.5	500 km
South China Sea — tension	war	5.5	600 km
Malacca — low congestion	natural	2.0	200 km
Suez Canal — draft limits	natural	4.5	150 km
Horn of Africa — piracy	piracy	6.0	450 km
Taiwan Strait — military	war	6.5	300 km

---

2. Radial-Basis Threat Scorer (replaces KNN)

New functions: `static_threat_score(lat, lon)` and `edge_threat_score(n1, n2, samples=5)`

Formula — RBF influence model:

```
threat(p) = Σ_z  severity_z · max(0,  1 − d(p, z) / radius_z)

clipped to [0, 10]
```

Where `d(p, z)` is the Haversine distance from point `p` to zone centre `z`. The `max(0, ...)` term gives **linear decay** inside the zone radius and **zero influence** outside it — a simple, interpretable piecewise-linear kernel.

For edges, the score is the **worst-case exposure** across 5 sampled midpoints:

```
edge_threat(u, v) = max{ threat(p_t) : t ∈ {0, 0.25, 0.5, 0.75, 1.0} }
```

This replaces the KNN distance-weighted average, which required live training data.

---

3. Plain Adjacency Dict (replaces NetworkX)

New function: `build_adjacency() → dict`

Returns a nested dict `adj[src][tgt] = {"dist": km, "base_risk": float}`.
Dijkstra and Yen's both operate directly on this dict. No third-party graph library required.

What Is Unchanged

Component	Status
Haversine distance formula	Unchanged
Edge weight formula `W(u,v) = α·threat + β·dist`	Unchanged (α=0.70, β=0.30)
Dijkstra algorithm	Unchanged
Yen's K-shortest loopless paths	Unchanged (bug in node restore fixed as side-effect of dropping NetworkX)
`XGBRiskScorer` class	Unchanged in structure; training sample size increased 2 000 → 3 000
XGBoost training label formula	Unchanged
14-dimensional feature vector	Unchanged
Risk class thresholds (LOW/MEDIUM/HIGH/CRITICAL)	Unchanged
`RoutePlan` dataclass	`knn_geo_threat_score` field removed; all other fields unchanged
`generate_warnings()`	Moved out of `VerificationLayer`, now a standalone function; logic identical
`CargoRiskPipeline.analyse()`	Unchanged in interface; internal KNN/news calls removed
JSON output format	Unchanged (minus `knn_threat_score` key)

Performance Comparison

Metric	v1.0	v2.0
XGBoost R^2 (held-out test)	0.9418	0.9689
Training samples	2 000	3 000
External API dependencies	1 (NewsAPI)	0
Third-party graph library	NetworkX	None
ML models at runtime	XGBoost + KNN	XGBoost only
Classes	5	2 (`XGBRiskScorer`, `CargoRiskPipeline`)
Lines of code	~1 050	~964
Cold-start time (no API key)	~2 s (fallback)	<1 s

Migration Guide

If you were using v1.0 directly:

```python
# v1.0
pipeline = CargoRiskPipeline(k_paths=5, news_query="shipping war piracy red sea")
pipeline.initialise()   # fetched news, verified, fitted KNN, trained XGB

# v2.0
pipeline = CargoRiskPipeline(k_paths=5)
pipeline.initialise()   # trains XGB only — no network calls
```

The `analyse()` method signature is identical. The returned `RoutePlan` objects no longer carry `knn_geo_threat_score`; use `threat_severity` instead, which is derived from `edge_threat_score()` via the RBF model.

Dependencies

Package	v1.0	v2.0
`xgboost`	✓	✓
`scikit-learn`	✓ (KNN + scaler + pipeline)	✓ (scaler only)
`numpy`	✓	✓
`networkx`	✓	✗ removed
`requests`	✓ (NewsAPI)	✗ removed
`pandas`	✓	✗ removed

Install for v2.0:

```bash
pip install xgboost scikit-learn numpy
```